

Partially Aggregatable Distributed Multi-Key Generation protocol

Rym KALAI¹[0000–0002–5301–329X], Wafa NEJI²[0000–0001–6692–2428], and Narjes BEN RAJEB³[0000–0003–2416–984X]

¹ LIPSIC Laboratory, Faculty of Sciences of Tunis, University of Tunis El Manar, Tunisia

² LIPSIC Laboratory, Higher Institute of Management of Bizerte (ISG Bizerte), University of Carthage, Bizerte, Tunisia

³ EPITA Reseach Lab (LRE), France

Abstract. In Publicly Verifiable Secret Sharing (PVSS) schemes, any participant can verify the validity of the shares distributed among the participants as well as the correctness of the recovered secret. Thus, the corrupted shares and misbehaving participants can be identified and there is no need for complaint management phase. Since there are less communication rounds, this contribution offers a positive impact in distributed protocols that are based on PVSS to decrease communication's complexity. An aggregatable PVSS permit to homomorphically combine several transcripts into a single transcript relative to the sum of their individual secrets.

In this paper, we propose a Distributed Multi-Key Generation (DMKG) protocol with partially aggregatable and publicly verifiable transcripts. The proposed protocol benefits from the efficiency of the complaint management strategy introduced by Kalai et al. in [20] and from the contribution of the publicly verifiable secret sharing (PVSS) scheme based on the enhanced SCRAPE PVSS presented by Gurkan et al. in [18].

Keywords: Publicly Verifiable Secret Sharing (PVSS) · Aggreagation · Distributed Multi-Key Generation (DMKG) protocol.

1 Introduction

Secret Sharing (SS) is a cryptographic technique that has been introduced by Shamir [31] and Blakley [6] in 1979. It enables one trusted party called dealer to split a secret into multiple shares and to distribute them among n participants. Individual shares are of no use on their own [21]. Furthermore, in threshold schemes, only a subgroup of the participants of size no less than t members such that ($2 \leq t < n$) can combine their shares together to recover the secret. Publicly Verifiable Secret Sharing (PVSS) is a cryptographic primitive proposed in [32] to overcome the weakness of threshold SS and to achieve security against cheating participants. Verifiability is the property of verifying the validity of the shares or the correctness of the recovered secret when the shares and the secret are not compromised. Stadler [32] designed the first PVSS. It enables a secret s to be shared between n participants through a publicly verifiable transcript of encrypted shares with a proof. This parameter allows any participant to check

the validity of the shares or the correctness of the recovered secret and to check that the dealer has behaved honestly. Even better, any third party can do this verification [3]. So, there is no need for complaint management phase to expose misbehaving participants. Indeed, this contribution has a positive and an outstanding impact in PVSS based protocols [13] especially distributed protocols to decrease communication's cost and complexity since fewer communication rounds are required. Aggregable PVSS makes it possible to homomorphically join several transcripts into a single aggregated transcript relative to the sum of their individual secrets.

In cryptographic system with SS scheme, the dealer is the unique participant that holds the secret and distributes the shares and whose corruption will break the security of the entire system. DKG protocols offer the solution to avoid this problem. The idea is to have many participants that cooperate together to generate and share a secret key without using a unique trusted party such that all honest participants will have valid shares of the unique secret key. It is equivalent to having n parallel secret sharing processes: Each participant is a dealer on a single sharing process. DKG protocol is an outstanding building block for many threshold cryptosystems such as data archive systems [35], public randomness beacons [33, 16] and Byzantine consensus [2, 36].

The term Distributed Multi-Key Generation (DMKG) protocol was presented in 2020 by Ma et al. in [22]. This is the same principle applied in DKG protocols, except that in DMKG protocols the participants handle l several secrets at the same time. It is equivalent to a distributed and parallel executions of a multi secret sharing (MSS) scheme where all the participants cooperate together and share several secrets in order to generate a public-private key pair.

In this article, we present a new DMKG protocol with an aggregatable PVSS scheme that benefits from the contributions of Gurkan et al. [18] which introduced an enhanced version of SCRAPE PVSS and applied it in a DKG protocol without disputation. Our contribution considerably reduces the communication complexity of our DMKG, simplifies the complaint management strategy previously defined in [20] and offers a public verification of the aggregated transcripts.

This paper is organized as follows: We start by presenting a state of art of DKG and DMKG protocols. Afterwards, we describe the important building block of our research which is the aggregatable DKG with the enhanced variant of SCRAPE PVSS introduced in [18]. After that, we present our new partially aggregatable DMKG with a simplified complaint management strategy. Then, we prove the security of the proposed contribution. Finally, we end this paper with a brief comparison of our proposed DMKG and some of the related works.

2 State of art

DKG protocol is an outstanding building block for many threshold cryptosystems. The first DKG without trusted party was presented in 1991 by Pedersen [28] for discrete log-based cryptosystems. It is based on Feldman VSS [12]. It was considered for many years as a central component to build a lot of cryptographic protocols such as thresh-

old ElGamal cryptosystem, Digital Signature Standard (DSS), and electronic voting. However, in 1999, Gennaro et al. [17] proved that in the presence of an adversary, the Pedersen's DKG does not guarantee a uniformly random distribution of generated keys. Gennaro et al. proposed a secure protocol based on Pedersen's VSS protocol [27] and Feldman's VSS protocol [12] but at the cost of lower efficiency. It uses private channels that hide malicious participants and requires an extra communication round to manage complaints. Despite that, several recent contributions used this protocol to construct threshold cryptosystems [37, 14, 34, 26]. Canetti et al. [7] improved the solution of Gennaro et al. to handle adaptive corruptions, but they also used private channels. Fouque and Stern [13] presented in 2001, a one-round DKG protocol without private channels. Instead of a VSS, it is based on a PVSS and publicly verifiable encryption and uses Paillier cryptosystem [25] to encrypt the shares. However, this leads to a more challenging key management process under different mathematical assumptions, to a slow key management and to a high communication costs. Neji et al. [24] proposed a more efficient DKG protocol as an extended version of Joint-Feldman DKG that guarantees a uniformly random distribution of generated keys. It removes the bias without using an additional secret sharing round. They also use public channels and introduced the disputation phase, a new strategy to manage the complaint among participants. However, this strategy requires additional consensus invocation. In PVSS schemes the main idea is that the dealer randomly and uniformly samples a t degree polynomial $f \in \mathbb{Z}_p$, commits to it via Feldman commitments [12] then computes encryptions of shares with Shamir threshold secret sharing. The other participants compute commitments to the shares $f(i)$ using Feldman commitments. Zero-knowledge proofs are used to prove that these commitments correspond to the encrypted received shares. Stadler [32] used the Fiat-Shamir heuristics in the random oracle model [3]. A variant of Stadler's PVSS is proposed by Schoenmakers [30] that improves the scheme's security and reduces it from the computational Diffie-Hellman assumption. Jhanwar [19] introduced a new PVSS that is based on pairing in the plain model that assures security under the Decisional Bilinear Square assumption and the multi-sequence of exponents Diffie-Hellman assumption. However, this results on quadratic computation cost. PVSS are used in distributed randomness beacons [29, 5, 4, 10] and DKG [13, 1, 18]. Over the years, several improved schemes have been presented. Cascudo et al. [8] designed SCRAPE PVSS scheme to overcome the problem of quadratic computation cost. The dealer commits to the polynomial evaluations $f(i)$ instead of the coefficients of the polynomial f and then publishes $g^{f(i)}$. Participants can verify that these commitments actually correspond to a t degree polynomial. Thus, the complexity reduces to $O(n)$.

Gurkan et al. [18] proposed an enhancement of the SCRAPE PVSS scheme to design a DKG protocol that requires from each participant P_i to broadcast n messages of $O(1)$ size and $\log n$ messages of size $O(n)$ where n is the number of participants cooperating in the DKG protocol. As mentioned in [10, 3, 11], the PVSS scheme proposed by Gurkan et al. [18] assumes hardness of the Symmetric External Diffie-Hellman (SXDH) problem for Type 3 pairing groups.

DMKG protocols apply the same idea proposed in DKG protocols except that they handle l several secrets at the same time. They solve the problem of the single trusted party in MSS schemes as each participant can be the dealer and all participants cooperate

together to share multiple secrets to generate a public-private key pair. In 2020 Ma et al. [22] introduced the first DMKG protocol to design BT-SOF : a Blockchain Traceable Scheme with Oversight Function. It implements identity tracing to overcome the flaws related to privacy protection in Blockchain applications. A regulator needs to trace the users' identities of the Blockchain data and needs the consent of the committee which is formed by n participants that cooperate together to generate the traceable public-private key pair $(sk, pk) = ((x_1, x_2, y_1, y_2, z), (C_1, C_2, C_3)) = ((x_1, x_2, y_1, y_2, z), (g_1^{x_1} g_2^{x_2}, g_1^{y_1} g_2^{y_2}, g_1^z))$. After that, the secret key is sent to the regulator to enable tracing. This protocol is built on the Gennaro's DKG protocol [17] to assure uniform distribution of generated keys. It uses private channels between participants who are not all honest. Some can distribute invalid shares and others may lie and claim to have received invalid shares. The use of private channels hides dishonest participants and leads to additional communications to manage the conflicts between participants and to difficulties to identify dishonest ones. In addition, whenever there is a complaint, the protocol requires participants to reveal their shares to prove their honesty which can lead a cheater to claim to have received an invalid share in order to disqualify the honest participant. Ma et al. [23] presented in 2021, a traceable scheme suitable for consortium Blockchain with leverage of [22].

Kalai et al. [20] introduced a new DMKG protocol built on the DMKG protocol of BT-SOF presented in [22]. It focused on the generation of the traceable secret key with a MSS scheme based on the Pedersen-VSS scheme [27] to distribute z_i and leverage the Franklin-Yung MSS scheme [15] to distribute (x_1, x_2, y_1, y_2) . The challenge is to prove the honesty of participants without revealing their shares and also to identify malicious ones. This DMKG protocol uses only public channels. There are no private channels between participants and all the shares of secrets are encrypted and sent over the public channel. In addition, the DMKG protocol presents a new strategy to manage complaints between participants that needs the contribution of all the participants. Thanks to this new strategy and to the use of public channels the protocol offers a precise and effective detection of malicious participants and avoids the honest ones to publicly reveal their shares of secrets. In fact, the protocol is able to eliminate participants that falsely pretend receiving invalid shares and participants that send invalid ones. In another hand, in this protocol there is no need to verify all broadcasted information of all participants. Each participant verifies his received shares and the complaint management strategy is executed whenever there is at least one complaint between two participants. But it turned out that this strategy leads to a considerable increase in the complexity and time of the protocol, particularly with the multi-secret context. For each complaint between 2 participants, all the participants cooperate together and each of the n participants verify the information received from all the other to finally be able to judge who is the dishonest one among the 2 participants in the conflict. The verification complexity is estimated to $O(kn^2)$ such that k is a security parameter.

In this paper, we propose a new DMKG protocol that leverage the efficient complaint management strategy proposed in [20] and benefits from the contribution of the enhanced version of SCRAPE PVSS introduced by Gurkan et al. in [18] to aggregate transcripts exchanged among participants. This contribution makes the transcripts pub-

licly verifiable, considerably simplifies the disputation phase and leads to a significant decrease in communication costs.

3 Aggregatable DKG protocol of Gurkan et al.

This section describe the DKG protocol of Gurkan et al. [18] build on an enhanced SCRAPE PVSS that is a variant of the pairing-based SCRAPE [8] that supports aggregation. As mentioned in [3], it is secure under the Symmetric External Diffie Hellman (SXDH) assumption for Type 3 pairings and it uses signatures of knowledge to efficiently aggregate PVSS transcripts. We note $bp = (p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, \hat{h}_1)$ a bilinear group description such that $\mathbb{G}_1, \mathbb{G}_2$ and $\mathbb{G}_T$ are groups with order divisible by the prime $p \in \mathbb{G}$, g_1 respectively $\hat{h}_1$ generates $\mathbb{G}_1$ respectively $\mathbb{G}_2$ and $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ is a (non-degenerate) bilinear map. All the participants use the same common reference string (CRS) that combine the following elements : bp a bilinear group description, $enc_i \in \mathbb{G}_2$ encryption keys with $i \in [1, n]$, $dec_i \in \mathbb{F}$ decryption keys where $enc_i = \hat{h}_1^{dec_i}$ with $i \in [1, n]$, vk_i verification keys with $i \in [1, n]$, n th roots of unity $\omega_i \in \mathbb{F}$ with $i \in [1, n]$, $\hat{u}_1 \in \mathbb{G}_2$ a random group element such that nobody knows $\log_{\hat{h}_1}(\hat{u}_1)$. Each participant deals a secret using the enhanced SCRAPE PVSS. In this DKG, to avoid the $O(n^2)$ verification complexity per participant as with other DKG, Gurkan et al. [18] leverage aggregation of SCRAPE PVSS transcripts : Each participant that verified more than one transcript can aggregate them to obtain a single verified transcript and sends it to another participant that will act in a similar way. In this way, it decreases the verification complexity per participant to $O(n \log^2 n)$. It should be noted that the aggregation can be achieved gradually and executed by any participant and that participant's contribution c_i can be integrated several times noted w_i in the final secret. Gurkan et al. [18] uses a signature of knowledge to permit to each participant P_i at the same time to sign its contribution and to prove knowledge of it to the final secret. P_i signs C_i using its secret key sk_i with corresponding verification key $verifi_i = g_1^{sk_i}$ as $\sigma_i = (\sigma_{i,1}, \sigma_{i,2}) = (Hash_{\mathbb{G}_2}(C_i)^{c_i}, Hash_{\mathbb{G}_2}(verifi_i, C_i)^{sk_i})$ such that $Hash_{\mathbb{G}_2}$ is a hash function that maps to $\mathbb{G}_2$. Then any verifier that has $verifi_i$ can verify σ_i as $e(C_i, Hash_{\mathbb{G}_2}(C_i)) = e(g_1, \sigma_{i,1}) \wedge e(verifi_i, Hash_{\mathbb{G}_2}(verifi_i, C_i)) = e(g_1, \sigma_{i,2})$. The different phases of the enhanced variant of SCRAPE PVSS are described in the following.

- Distribution phase A dealer share a secret $\hat{u}_1^{a_0} \in \mathbb{G}_2$ such that $a_0 \in \mathbb{F}$ then randomly chooses a polynomial $f(x) = a_0 + a_1x + \dots + a_tx^t$ such that $f(0) = a_0$, after that commits to it via Feldman [12] $F_i = g_1^{a_i}$ for $i = 0, \dots, t$. Next, the dealer computes the PVSS transcript that consists of : the shares such that $\hat{h}_1^{f(\omega_i)}$, the Feldman commitments such that $A_i = g_1^{f(\omega_i)}$ and the shares' encryptions $enc_i^{f(\omega_i)}$.

- Verification phase P_i verifies, via Lagrange interpolation in the exponent, if the Feldman commitments A_i to the shares $f(\omega_i)$ are compatible with the Feldman commitments to $f(x)$. Next, P_i verifies their encryption of $f(\omega_i)$ against A_i . This phase enables the participant to check whether the PVSS transcript is a correct sharing of $\hat{u}_1^{a_0}$ and to guarantees that encrypted shares are the evaluations of the committed polynomial f .

- **Aggregation phase** This is the most important phase in the PVSS of Gurkan et al. [18]. The aggregation benefits from the homomorphism of Feldman commitments and of the encryption scheme. The scheme consider as input 2 PVSS transcripts $pvss_1$ for polynomial f_1 and $pvss_2$ for polynomial f_2 and outputs an aggregated transcript for their sum $f_1 + f_2$ as follows : for $b \in \{1, 2\}$ and $i \in [0, t]$ $F_{b,i} = g_1^{a_{b,i}}$ will be aggregated such that $F_i = F_{1,i}F_{2,i} = g^{a_{1,i}+a_{2,i}}$ is an Feldman commitment to $f_1 + f_2$. In the same way, $A_i = A_{1,i}A_{2,i} = g^{(f_1+f_2)(\omega_i)}$ as the share commitments. Then, the encryptions $enc_i^{(f_1+f_2)(\omega_i)} = enc_i^{f_1(\omega_i)} enc_i^{f_2(\omega_i)}$.

- **Reconstruction phase** Note that S is the set of honest participants. First, P_i decrypt its share $\hat{A}_i = \hat{Y}_i^{dec_i^{-1}} = (enc_i^{f(\omega_i)})^{dk_i^{-1}} = \hat{h}_1^{f(\omega_i)}$ for $i \in [1, n]$. If there is no supplied share from P_i , then P_i is disqualified. Then, each supplied decrypted share is checked as $e(A_i, \hat{h}_1) \stackrel{?}{=} e(g_1, \hat{A}_i)$. If it fails, then P_i is disqualified. The secret $sk = \prod_{i \in S} \hat{A}_i^{\mathcal{L}_{S,i}(0)}$ can be recovered by any set S of size at least t where $\mathcal{L}_{S,i}(X)$ is a Lagrange polynomial that is equal to 1 at ω_i and equal to 0 at $\omega_j \in S$ for $i \neq j$.

4 Proposed Partially Agregatable DMKG Protocol

The traceable public-private key pair $(sk, pk) = ((x_1, x_2, y_1, y_2, z), (C_1, C_2, C_3)) = ((x_1, x_2, y_1, y_2, z), (g_1^{x_1} g_2^{x_2}, g_1^{y_1} g_2^{y_2}, g_1^z))$ is generated by the committee through the DMKG protocol. In [20], for the traceable secret key generation, the VMSS scheme is based on the Pedersen-VSS scheme to distribute z and leverage the Franklin-Yung MSS scheme to distribute (x_1, x_2, y_1, y_2) . In this present contribution, we use the enhanced SCRAPE PVSS presented by Gurkan et al. [18] to distribute z . Since any third party can perform the verification, so there is no need for complaint management phase. Thus, this improvement leads to a considerable simplification in the complaint management strategy introduced in [20].

4.1 Communication And Adversary Model

We suppose that there are n probabilistic polynomial time participants $P_1, \dots, P_n$, a common broadcast channel between them in a fully synchronous network and a static adversary that chooses the corrupted participants at the beginning of the protocol and can corrupt at most $(t - 1)$ participants such that $(t - 1) < \frac{n}{2}$.

4.2 Generation of z

In this section, we describe our contribution based on the enhanced SCRAPE PVSS presented by Gurkan et al. [18] to distribute z component for the traceable secret key generation. In addition, we construct the set Q_{agg} a set of dishonest participants that have been reported whenever a participant P_i encounters non-compliance and inconsistency in the verification phase. At the end, Q_{agg} is made up of all dishonest participants that have been identified in the generation of the component z . This set will be taken into consideration during the simplified complaints management phase of the VMSS

described in the next section.

Phase 1 : Initialisation Phase All the participants use the same SCRAPE Common Reference String (CRS) described in details in 3. Each participant P_i has a decryption key $dec_i \in \mathbb{F}$ for enc_i where $enc_i = \hat{h}_1^{dec_i}$ and a secret key sk_i for $verifi_i$ where $verifi_i = g_1^{sk_i}$

Phase 2 : Distribution Phase This phase is carried out in several steps as follows:

Step 2.a Participant P_i chooses at random $z_i \in \mathbb{F}$

Step 2.b Participant P_i randomly chooses a polynomial $h_i(x)$ of degree t such that $h_i(x) = a''_{i0} + a''_{i1}x + \dots + a''_{it}x^t$ where $h_i(0) = z_i = a''_{i0}$,

Step 2.c Participant P_i computes $Z_i = g_1^{z_i}$, commits to it via Feldman and broadcast it to other participants.

Step 2.d Participant P_i computes the shares $h_i(\omega_j)$ for $j \in [1, n]$

Step 2.e Participant P_i computes a $pvss_T$ that includes $H_{ik} = g_1^{a''_{ik}}$ for $k \in [0, t]$, $\hat{u}_{i2} = \hat{u}_1^{z_i}$, a vector w_i where $w_{ij} = 1$ if $i = j$, otherwise $w_{ij} = 0$, $A_{ij} = g_1^{h_i(\omega_j)}$ Feldman commitments, $Y_{ij} = enc_j^{h_i(\omega_j)}$ encryption of each share, a vector σ_i where $\sigma_{ij} = (Hash_{G_2}(Z_i)^{z_i}, Hash_{G_2}(verifi_i, Z_i)^{sk_i})$ if $i = j$, otherwise $\perp$.

Step 2.f As mentioned in [18], each transcript $pvss_T$ is linked with the information related to the public-verifiability to obtain the transcript $T = ((Z_1, \dots, Z_n), (w_1, \dots, w_n), (\sigma_1, \dots, \sigma_n), pvss_T)$ that includes: $Z_i = g_1^{z_i}$ the commitment to each participant's contribution z_i , the weights w_i of z_i , the signature of knowledge σ_i of z_i and a PVSS transcript for $z = \sum_{i=1}^n w_i z_i$.

Step 2.g Participant P_i gossips its transcript according to the gossip phase described in [18].

Phase 3 : Verification Phase In this phase, each participant P_i verifies the received transcripts T according to the description below. In case that two transcripts verify then the participant P_i aggregate these transcripts according to the description in *Step 4*. First of all, each participant P_i checks if the received $pvss_T$ is a correct sharing of the secret. It checks if the Feldman commitments A_{ij} to the shares $h_j(\omega_i)$ is in accordance with the Feldman commitment to $h_j(x)$ via Lagrange interpolation in the exponent. After that P_i checks the Feldman commitments A_{ij} against $enc_j^{h_j(\omega_i)}$ the encryption of $h_j(\omega_i)$. Next, we can check the signature of knowledge σ_i of each z_i that has $w_i \neq 0$ in T . Then, we verify that $\prod_{i=1}^n Z_i^{w_i} = H_0$. So, if all checks verify, we consider that every participant P_i that has $w_i \neq 0$ in T is a honest participant else we consider that it is dishonest and added to Q_{agg} the set of reported dishonest participants.

Phase 4 : Aggregation Phase In this step, the main idea is aggregating two transcripts T_1 and T_2 such that $T_1 = ((Z_{11}, \dots, Z_{1n}), (w_{11}, \dots, w_{1n}), (\sigma_{11}, \dots, \sigma_{1n}), pvss_T_1)$, $T_2 = ((Z_{21}, \dots, Z_{2n}), (w_{21}, \dots, w_{2n}), (\sigma_{21}, \dots, \sigma_{2n}), pvss_T_2)$ into a unique transcript $T = ((Z_1, \dots, Z_n), (w_1, \dots, w_n), (\sigma_1, \dots, \sigma_n), pvss_T)$

Step 4.a We aggregate $pvss_T_1$ for polynomial h_1 and $pvss_T_2$ for polynomial h_2 into a unique transcript $pvss_T$ for $h_1 + h_2$. It exploits the homomorphism of encryption scheme and of Feldman commitments such that $H_i = H_{1i}H_{2i}$ a Feldman commitments to $h_1 + h_2$ for $i = 1, \dots, t$; then for $i = 1, \dots, n$, $A_i = A_{1i}A_{2i} = g^{(h_1+h_2)(\omega_i)}$ a share commitments; $\hat{Y}_i = \hat{Y}_{1i}\hat{Y}_{2i} = enc_i^{(h_1+h_2)(\omega_i)} = enc_i^{h_1(\omega_i)} enc_i^{h_2(\omega_i)}$ the encryptions. Next, we

aggregate $\hat{u}_i = \hat{u}_{12}\hat{u}_{22}$; $w_i = w_{1i} + w_{2i}$; $\sigma_i = \sigma_{1i}$ if $\sigma_{1i} \neq \perp$ otherwise $\sigma_i = \sigma_{2i}$; $Z_i = Z_{1i}$ if $Z_{1i} \neq \perp$ otherwise $Z_i = Z_{2i}$.

Step 4.b P_i gossips the aggregated transcript T .

Step 4.c In each transcript $(w_1, \dots, w_n)$ indicates the number of contributions of every participant. When P_i gets a complete transcript, meaning transcript with a size $k \geq t$ of $w_j \neq 0$, then P_i broadcast it as a candidate final transcript and the Aggregation ends here for the participant P_i .

Step 4.d This step is executed whenever there is a complete transcript that has been broadcast by a participant. If there are many candidate transcripts, we select the one with the lowest bit count in $(\prod_{i=1}^n Z_i^{\omega_i}, \prod_{i=1}^n \hat{u}_{i2}^{\omega_i})$ as the final transcript.

Phase 5 : Reconstruction Phase Since the selected transcript T is actually an extended SCRAPE PVSS transcript as described by Gurkan et al. [18], we use the same reconstruction phase of section 3.

4.3 Simplified DMKG Protocol

Our previous DMKG protocol [20] introduced an efficient complaint management strategy to avoid the participants from publicly revealing their shares of secrets whenever there is a conflict between them. It is true that disputation was an effective way of managing conflicts between participants, identifying malicious participants and eliminating them, but this strategy added an extra complexity to the protocol. Indeed, each of the two participants in conflict must prove his innocence to all the other participants by regenerating information related to all the components of the private key. Since the z component of the private key has been aggregated and generated in a publicly verifiable way, we no longer need to use a complaint management strategy for this part. As a result, the complaint management phase is simplified to eliminate the whole complaint management part concerning z which considerably reduces the number of exchanged messages between participants and communication time. We focus only on exposing misbehaving participants for (x_1, x_2, y_1, y_2) generation. In addition, we take into consideration the set Q_{agg} of dishonest participants already identified in section 4.2. Let denote p and q two large prime numbers where $q|(p-1)$; $\mathbb{Z}_p$ a group of order p and $\mathbb{Z}_q$ a group of order q . The exponential operation performs modulo p operation such that g^x refers to $g^x \bmod p$, where $g \in \mathbb{Z}_p$ and $x \in \mathbb{Z}_q$. The simplified DMKG protocol is done as follows:

Phase 1 : Distribution Phase As a dealer P_i randomly chooses $x_{1i}, x_{2i}, y_{1i}, y_{2i} \in \mathbb{Z}_q$; $\beta_{0i}, \beta_{1i}, \beta_{2i}, \beta_{3i}, \beta_{4i} \in \mathbb{Z}_q$ and 4 polynomials $f(x), f'(x), g(x)$ and $g'(x)$ of degree t such that $f_i(x) = a_{i0} + a_{i1}x + \dots + a_{it}x^t$ where $f_i(-1) = x_{1i}$, $f_i(-2) = y_{1i}$, $f'_i(x) = b_{i0} + b_{i1}x + \dots + b_{it}x^t$ where $f'_i(-1) = \beta_{1i}$, $f'_i(-2) = \beta_{2i}$, $g_i(x) = a'_{i0} + a'_{i1}x + \dots + a'_{it}x^t$ where $g_i(-1) = x_{2i}$, $g_i(-2) = y_{2i}$, $g'_i(x) = b'_{i0} + b'_{i1}x + \dots + b'_{it}x^t$ where $g'_i(-1) = \beta_{3i}$, $g'_i(-2) = \beta_{4i}$. Next, P_i broadcasts $CM_{ik} = g_1^{a_{ik}}.h_1^{b_{ik}}.g_2^{a'_{ik}}.h_2^{b'_{ik}}$ for $k \in [0, t]$. After that, P_i computes the shares $sf_{ij} = f_i(j)$, $sf'_{ij} = f'_i(j)$, $sg_{ij} = g_i(j)$, $sg'_{ij} = g'_i(j)$ for $j \in [1, n]$. Note that, each participant P_i has a public-private key pair (sk_{P_i}, pk_{P_i}) to encrypt and decrypt the shares. So, P_i computes for $j \in [1, n]$: $enc_{sf_{ij}} = sf_{ij}(pk_{P_j})^{sf_{ij}} = sf_{ij}(g^{sk_{P_j}})^{sf_{ij}}$, $enc_{sf'_{ij}} = sf'_{ij}(pk_{P_j})^{sf'_{ij}} = sf'_{ij}(g^{sk_{P_j}})^{sf'_{ij}}$, $enc_{sg_{ij}} = sg_{ij}(pk_{P_j})^{sg_{ij}} = sg_{ij}(g^{sk_{P_j}})^{sg_{ij}}$, $enc_{sg'_{ij}} =$

$$sg'_{ij}(pk_{P_j})^{sg'_{ij}} = sg'_{ij}(g^{sk_{P_j}})^{sg'_{ij}}.$$

Finally, P_i sends publicly the encrypted shares $(enc_{sf_{ij}}, enc_{sf'_{ij}}, enc_{sg_{ij}}, enc_{sg'_{ij}})$ to P_j for $j \in [1, n]$.

Phase 2 : Verification Phase This phase is executed as follows:

Step 2.a To decrypt the received shares from P_i , each participant P_j uses his private key sk_{P_j} as follows $sf_{ij} = enc_{sf_{ij}}[(g^{sf_{ij}})^{sk_{P_j}}]^{-1}$, $sf'_{ij} = enc_{sf'_{ij}}[(g^{sf'_{ij}})^{sk_{P_j}}]^{-1}$, $sg_{ij} = enc_{sg_{ij}}[(g^{sg_{ij}})^{sk_{P_j}}]^{-1}$, $sg'_{ij} = enc_{sg'_{ij}}[(g^{sg'_{ij}})^{sk_{P_j}}]^{-1}$.

Step 2.b Each participant P_j verify Eq. 1 to validate the set of received shares. If it verify, then P_j accepts the shares. Otherwise, P_j complaints against P_i .

$$g_1^{sf_{ij}}.h_1^{sf'_{ij}}.g_2^{sg_{ij}}.h_2^{sg'_{ij}} \stackrel{?}{=} \prod_{k=0}^t (CM_{ik})^{j^k} \quad (1)$$

Phase 3 : Simplified Complaint Management Phase It should be noted that the disputation is only executed if there is a complaint from a participant P_j against a participant P_i . First of all, we check whether P_i is present in Q_{agg} the set of dishonest participants reported during aggregation. If this is the case, then P_i is disqualified. Otherwise, we verify if the number of complaints against P_i is $> (t - 1)$ in the *Step 2.b* then we consider P_i as disqualified. After that, each participant that is not disqualified and is not present in Q_{agg} will be added to Q_{temp} the set of temporarily qualified participants. At the end, we accept all the complaints against a participant P_i that is already disqualified. Otherwise, both P_i and P_j have to demonstrate their honesty to all the other participants. Let's consider the set $Q = \{Q_{temp} \setminus \{P_i, P_j\}\}$.

Step 3.a P_j randomly chooses $S'_j = \{a_j, b_j, a'_j, b'_j\}$ and publishes $S''_j = \{g_1^{a_j}, g_2^{a'_j}, h_1^{b_j}, h_2^{b'_j}\}$

Step 3.b Each $P_k \in Q$ randomly chooses $S'_k = \{a_k, b_k, a'_k, b'_k\}$ and publishes $S''_k = \{g_1^{a_k}, g_2^{a'_k}, h_1^{b_k}, h_2^{b'_k}\}$.

Step 3.c P_i computes and publishes $\lambda_1 = sf_{ij}[g_1^{a_j} \cdot \prod_{k=1; P_k \in Q}^n g_1^{a_k}]^{sf_{ij}}$, $\lambda_2 = sf_{ij}(g_1^{a_j})^{sf_{ij}}$, $\lambda'_1 = sg_{ij}[g_2^{a'_j} \cdot \prod_{k=1; P_k \in Q}^n g_2^{a'_k}]^{sg_{ij}}$, $\lambda'_2 = sg_{ij}(g_2^{a'_j})^{sg_{ij}}$, $\gamma_1 = sf'_{ij}[h_1^{b_j} \cdot \prod_{k=1; P_k \in Q}^n h_1^{b_k}]^{sf'_{ij}}$, $\gamma_2 = sf'_{ij}(h_1^{b_j})^{sf'_{ij}}$, $\gamma'_1 = sg'_{ij}[h_2^{b'_j} \cdot \prod_{k=1; P_k \in Q}^n h_2^{b'_k}]^{sg'_{ij}}$, $\gamma'_2 = sg'_{ij}(h_2^{b'_j})^{sg'_{ij}}$.

Step 3.d Each $P_k \in Q$ publishes his set S'_k . If it not coherent with his set S''_k , then it will be disqualified and removed from Q , and the process resumes at *Step 3.b*.

Step 3.e Each $P_k \in Q$ computes $\alpha = \frac{\lambda_1}{\lambda_2}$, $\alpha' = \frac{\lambda'_1}{\lambda'_2}$, $\beta = \frac{\gamma_1}{\gamma_2}$, $\beta' = \frac{\gamma'_1}{\gamma'_2}$ and $r = \sum_{k=1}^n a_k$, $r' = \sum_{k=1}^n a'_k$, $t = \sum_{k=1}^n b_k$, $t' = \sum_{k=1}^n b'_k$. Next, P_k computes: $\alpha^{\frac{1}{r}} = g_1^{sf_{ij}}$, $\alpha'^{\frac{1}{r'}} = g_2^{sg_{ij}}$ and $\beta^{\frac{1}{t}} = h_1^{sf'_{ij}}$, $\beta'^{\frac{1}{t'}} = h_2^{sg'_{ij}}$. P_k verifies Eq. 2. If it checks, then the protocol continues. Otherwise, $(sf_{ij}, sf'_{ij}, sg_{ij}, sg'_{ij})$ is not accepted and P_k identifies P_i as dishonest and the protocols ends.

$$\alpha^{\frac{1}{r}}.\beta^{\frac{1}{t}}.\alpha'^{\frac{1}{r'}}.\beta'^{\frac{1}{t'}} \stackrel{?}{=} \prod_{k=0}^t (CM_{ik})^{j^k} \quad (2)$$

Step 3.f Participant P_j computes: $sf_{ij} = \frac{\lambda_2}{(g_1^{sf_{ij}})^{a_j}}$, $sg_{ij} = \frac{\lambda'_2}{(g_2^{sg_{ij}})^{a'_j}}$ and $sf'_{ij} = \frac{\gamma_2}{(h_1^{sf'_{ij}})^{b_j}}$,

$sg'_{ij} = \frac{\gamma'_j}{(h_2^{sg'_{ij}})^{b'_j}}$ then verifies if they check Eq.1. If it fails, P_j sends S'_j to all participants in Q . Else, P_j confirms to all participants in Q that it has a valid shares and the protocol ends.

Step 3.g Each $P_k \in Q$ verifies the validity of the set S'_j sent by P_j by checking the coherence between the values $(g_1^{a_j}, g_2^{a'_j}, h_1^{b_j}, h_2^{b'_j})$ published by P_j in *step 3.a* and the values of S'_j published by P_j in *step 3.f*. If it fails, P_k identifies P_j as dishonest and the protocol ends. Else, the protocol continues.

Step 3.h Each $P_k \in Q$ computes $(g_1^{sf_{ij}})^{a_j}, (g_2^{sg_{ij}})^{a'_j}, (h_1^{sf'_{ij}})^{b_j}, (h_2^{sg'_{ij}})^{b'_j}$ and $e_1 = \frac{\lambda_2}{(g_1^{sf_{ij}})^{a_j}}, e'_1 = \frac{\lambda'_2}{(g_2^{sg_{ij}})^{a'_j}}, e_2 = \frac{\gamma_2}{(h_1^{sf_{ij}})^{b_j}}, e'_2 = \frac{\gamma'_2}{(h_2^{sg'_{ij}})^{b'_j}}$. Then, P_k verifies if the equations in *Step 3.c* are correct with the values $e_1 = sf_{ij}, e'_1 = sg_{ij}, e_2 = sf'_{ij}, e'_2 = sg'_{ij}$. If it checks, then P_k identifies that P_j is dishonest, else, P_i is dishonest. Finally P_k broadcasts the vote about the identified dishonest participant : P_i or P_j .

Step 3.i From the votes sent by each participants P_k , we can determine the dishonest participant between P_i and P_j . Thus, all dishonest participants are identified and eliminated.

Phase 4 : Recovery Phase We consider $QUAL$ as the set of non disqualified participants. Each participant $P_i \in QUAL$ computes the shares: $sf_i = \sum_{j \in QUAL} sf_{ji} \text{mod } q$, $sf'_i = \sum_{j \in QUAL} sf'_{ji} \text{mod } q$, $sg_i = \sum_{j \in QUAL} sg_{ji} \text{mod } q$, $sg'_i = \sum_{j \in QUAL} sg'_{ji} \text{mod } q$.

5 Security

We rely on the security requirements for DMKG protocols defined in [9] in the presence of an adversary that can corrupts at most $t - 1$ participants such that $t - 1 < \frac{n}{2}$. These requirements are correctness and secrecy.

5.1 Security Requirements

Correctness The correctness of the protocol consists of the following elements:

C1 The same unique secret key $sk = (x_1, x_2, y_1, y_2, z)$ is defined by any set of t shares supplied by honest participants.

C2 There is an efficient algorithm that takes on input the public information generated by the DMKG protocol and the n shares submitted by participants and that outputs the unique secret key sk even if up to $t - 1$ invalid shares are submitted by malicious participants.

C3 All honest participants have the same public key $pk = (c_1, c_2, c_3) = (g_1^{x_1} g_2^{x_2}, g_1^{y_1} g_2^{y_2}, g_1^z)$, such that (x_1, x_2, y_1, y_2, z) is the unique secret key guaranteed by (C1).

C4 The values of the secret key components $sk = (x_1, x_2, y_1, y_2, z)$ are uniformly distributed in $\mathbb{Z}_q$.

Secrecy The DMKG protocol verifies the secrecy requirement if the adversary cannot get sk , or any information on sk except for the public key pk . The shared private key sk is a confidential information that should not be found by any party that is unauthorized to get it. The secrecy of the DMKG can be formally expressed by a simulator. We

keep the same simulation as in [22]. It proves that an adversary that influences and corrupts a certain number of participants without exceeding $(t - 1)$, cannot compute the value of a secret key sk . As input, the simulator gets the public key pk and outputs a distribution that generates pk and that is indistinguishable from the adversary's view of a real execution of the DMKG protocol. Thus, the adversary cannot compute the value of the secret key sk .

5.2 Security Proofs

In this section, we show that our DMKG protocol satisfies the security requirements defined in section 5.1. First of all, we need to present the following lemmas about the VMSS scheme in [22].

Lemma 1. *Any t shares of the honest participants can reconstruct the secret s in the presence of an adversary that corrupts at most $t - 1$ participants. If the dealer is honest, then all shares held by honest participants can interpolate to a unique polynomial of degree t .*

Lemma 2. *Any t shares of the honest participants can reconstruct the secret set $S = s_1, \dots, s_l$ in the presence of an adversary that corrupts at most $t - l$ participants. If the dealer is honest, then all shares held by honest participants can interpolate to a unique polynomial of degree t .*

Correctness Proof

Theorem 1. *Our DMKG protocol satisfies the security requirement (C1). All shares submitted by a set of honest participants define the same private key sk .*

Proof. At the end of the *Complaint Management* phase, if the participant P_i is not disqualified, which means that $P_i \in \text{QUAL}$ and that as a dealer, he has correctly performed the VMSS scheme. We rely on Lemma 1 and Lemma 2. Then, all honest participants have valid shares of P_i that they use to compute the polynomials $f_i(x)$, $g_i(x)$ and $h_i(x)$ where $f_i(-1) = x_{1i}$, $f_i(-2) = y_{1i}$, $g_i(-1) = x_{2i}$ and $g_i(-2) = y_{2i}$. For this purpose, for a set Q of t valid shares, a unique value x_{1i} (respectively x_{2i} , y_{1i} , y_{2i}) is computed with the Lagrange interpolation as $x_{1i} = \sum_{j \in Q} \lambda_{1j} s_{fij}$, $x_{2i} = \sum_{j \in Q} \lambda_{1j} s_{gij}$, $y_{1i} = \sum_{j \in Q} \lambda_{2j} s_{fij}$, $y_{2i} = \sum_{j \in Q} \lambda_{2j} s_{gij}$ where $\lambda_{1j} = \prod_{k \in Q, k \neq j} \frac{-1-k}{j-k}$, $\lambda_{2j} = \prod_{k \in Q, k \neq j} \frac{-2-k}{j-k}$ are the coefficient of Lagrange. Thus, from these shares x_1 (respectively x_2 , y_1 , y_2) can be generated as $x_1 = \sum_{i \in \text{QUAL}} x_{1i} = \sum_{i \in \text{QUAL}} \sum_{j \in Q} \lambda_{1j} s_{fij} = \sum_{j \in Q} \lambda_{1j} \sum_{i \in \text{QUAL}} s_{fij} = \sum_{j \in Q} \lambda_{1j} s_{fj}$ (respectively $x_2 = \sum_{j \in Q} \lambda_{1j} s_{gj}$, $y_1 = \sum_{j \in Q} \lambda_{2j} s_{fj}$, $y_2 = \sum_{j \in Q} \lambda_{2j} s_{gj}$). For any authorized set Q of participants, the private key $sk = (x_1, x_2, y_1, y_2, z)$ is unique.

Theorem 2. *Our DMKG protocol satisfies the security requirement (C2). Only valid shares of honest participants who check successfully the shares verification are used to reconstruct the private key sk .*

Proof. The validity of the shares (s_{fi}, s_{gi}) submitted by the participant P_i with $i \in \text{QUAL}$ can be checked as follows: $g_1^{s_{fi}} g_2^{s_{gi}} = g_1^{\sum_{i \in \text{QUAL}} s_{fij}} g_2^{\sum_{i \in \text{QUAL}} s_{gij}} = \prod_{i \in \text{QUAL}} g_1^{s_{fij}} g_2^{s_{gij}} = \prod_{i \in \text{QUAL}} \prod_{k=0}^t (A_{ik})^{j^k}$. Therefore, only valid shares (s_{fi}, s_{gi}) submitted by honest participants who passed successfully the check are used.

Theorem 3. *Our DMKG protocol satisfies the security requirement (C3). After the Complaint Management phase, all qualified participants compute the same value of the public key $pk = (c_1, c_2, c_3) = g_1^{x_1} g_2^{x_2}, g_1^{y_1} g_2^{y_2}, g_1^z$, where $sk = (x_1, x_2, y_1, y_2, z)$ is the unique secret key guaranteed by Theorem 1.*

Proof. Since we did not modify the generation of the public key used in [22], we use the same proof of Theorem 3 to prove that all qualified participants from the Complaint Management phase compute the same value of the public key pk . More details in [22].

Theorem 4. *Our DMKG protocol satisfies the security requirement (C4). The values x_1, x_2, y_1 and y_2 of the secret key sk are uniformly distributed in $\mathbb{Z}_q$.*

Proof. At the end of the Complaint Management phase, only honest participants are in the set $QUAL$. Each participant $P_i, i \in QUAL$ has already selected a secret value x_{1i} (respectively x_{2i}, y_{1i}, y_{2i}) randomly in $\mathbb{Z}_q$. As x_1 (respectively x_2, y_1, y_2) is defined as the sum of all x_{1i} (respectively x_{2i}, y_{1i}, y_{2i}) then it can be guaranteed that x_1 (respectively x_2, y_1, y_2) is randomly chosen in $\mathbb{Z}_q$. At the end of the Distribution phase of our DMKG protocol, these values are already chosen and uniformly distributed via VMSS protocol. Neither the values x_{1i} (respectively x_{2i}, y_{1i}, y_{2i}) nor the set $QUAL$ change later. Thereby, as $x_1 = \sum_{i \in QUAL} x_{1i}$, then x_1 is uniformly distributed in $\mathbb{Z}_q$. In the same way, x_2, y_1 and y_2 are also uniformly distributed.

Secrecy Proof The secrecy of the DMKG is formally expressed by a simulator. Since we have kept the same simulator used in [22], we also keep the same proof of secrecy. We hence refer the reader to the aforementioned publication for further details.

6 Comparison and Discussion

The comparison of our aggregatable DMKG protocol with the related works is reviewed in Table 1. It is based on the following criteria: first, the communication model: private or public channels, second, the verification complexity, third, the complaint management efficiency based on the fact that honest participants are not forced to divulge their shares and on the fact that dishonest participants are identified and disqualified, fourth, the public verifiability of the secret sharing scheme, fifth, whether the transcript is aggregatable or not and finally the context of secret sharing : multi-key generation or not. BTSOF's DMKG uses private channels to send shares to participants and a broadcast channel. As a result, each participant will have to check the received shares of the secret as well as the proof of verifiability of these shares. The complexity is therefore $O(n^2)$ per participant. On the other hand, this DMKG uses an interactive and non-publicly verifiable secret sharing scheme which requires significant communication time and a complaint management strategy to identify dishonest participants. However, the used strategy forces conflicting participants to reveal their shares, which creates a major security flaw and makes the complaints management strategy ineffective and unable to identify all dishonest participants.

In [20], Kalai et al. used only public channels. Each participant checks its received shares of the secret, which implies a complexity $O(kn)$ per participant. But, if there is

a complaint between two participants, each participant in the conflict management will check all the data sent by all the participants which results in a quadratic complexity for the verifications carried out during the complaint management strategy. This complexity increases with the number of complaints to manage.

In our proposed DMKG protocol, the idea is to simplify the complaint management strategy and reduce the probability of its execution. Indeed, we used a PVSS which makes it possible to publicly verify the shares and not have to resort to complaint management. Furthermore, when distributing the component z , we identified a set of dishonest participants and constructed Q_{agg} . This set is taken into consideration before executing the complaints management phase. Indeed, all complaints against participants in Q_{agg} are accepted, thus avoiding any unnecessary additional complaints management. On the other hand, for the distribution of the component z , the use of the improved version of SCRAPE PVSS made it possible to reduce the cost of verification to $O(n \log n)$.

Table 1. Comparison of our Aggregatable DMKG protocol with related work.

Criteria	Communication model	Verification complexity	Complaint management efficiency	Publicly verifiable	Aggregatable	Multi-key generation
BTSOF [23]	public + private	$O(kn^2)$	no	no	no	yes
Kalai et al.[20]	public	$O(kn^2)$	yes	no	no	yes
Gurkan et al.[18]	public	$O(kn \log n)$	no need	yes	yes	no
our DMKG	public	$O(kn \log n)$	yes	partially	partially	yes

7 Conclusion

In this paper, we presented our partially aggregatable DMKG protocol with a simplified complaint management strategy based on our previous DMKG introduced in [20]. We leverage the enhanced SCRAPE PVSS scheme described by Gurkan et al. [18] that allows to share a secret among n participants through a publicly verifiable transcript and moreover to homomorphically combine several transcripts into a single aggregate transcript result of their sum. Indeed, the participants cooperate together to generate, verify and aggregate shares in order to avoid the use of a complaint management strategy that increases the complexity of the protocol. The implementation of our contribution will be the subject of a new article which will be supported by experimental results. We also plan to expand the use of aggregation in our DMKG to design a totally aggregatable publicly verifiable multi-secret sharing scheme to benefit from the contribution of aggregation in the entire sk component. The proposed DMKG protocol can be seen as a basic contribution that can be extended and applied in other contexts such as identity-based cryptography and electronic voting.

References

1. Abraham, I., Jovanovic, P., Maller, M., Meiklejohn, S., Stern, G., Tomescu, A.: Reaching consensus for asynchronous distributed key generation. In: *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing*. pp. 363–373 (2021)
2. Abraham, I., Malkhi, D., Spiegelman, A.: Asymptotically optimal validated asynchronous byzantine agreement. In: *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing*. pp. 337–346 (2019)
3. Bacho, R., Loss, J.: Adaptively secure (aggregatable) pvss and application to distributed randomness beacons. In: *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. pp. 1791–1804 (2023)
4. Bhat, A., Shrestha, N., Kate, A., Nayak, K.: Optrand: Optimistically responsive distributed random beacons. *Cryptology ePrint Archive* (2022)
5. Bhat, A., Shrestha, N., Luo, Z., Kate, A., Nayak, K.: Randpiper—reconfiguration-friendly random beacons with quadratic communication. In: *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. pp. 3502–3524 (2021)
6. Blakley, G.R.: Safeguarding cryptographic keys. In: *Managing Requirements Knowledge, International Workshop on*. pp. 313–313. IEEE Computer Society (1979)
7. Canetti, R., Gennaro, R., Jarecki, S., Krawczyk, H., Rabin, T.: Adaptive security for threshold cryptosystems. In: *Advances in Cryptology—CRYPTO’99: 19th Annual International Cryptology Conference Santa Barbara, California, USA, August 15–19, 1999 Proceedings 19*. pp. 98–116. Springer (1999)
8. Cascudo, I., David, B.: Scrape: Scalable randomness attested by public entities. In: *International Conference on Applied Cryptography and Network Security*. pp. 537–556. Springer (2017)
9. Cramer, R., Shoup, V.: A practical public key cryptosystem provably secure against adaptive chosen ciphertext attack. In: *Annual international cryptology conference*. pp. 13–25. Springer (1998)
10. Das, S., Krishnan, V., Isaac, I.M., Ren, L.: Spurt: Scalable distributed randomness beacon with transparent setup. In: *2022 IEEE Symposium on Security and Privacy (SP)*. pp. 2502–2517. IEEE (2022)
11. Das, S., Yurek, T., Xiang, Z., Miller, A., Kokoris-Kogias, L., Ren, L.: Practical asynchronous distributed key generation. In: *2022 IEEE Symposium on Security and Privacy (SP)*. pp. 2518–2534. IEEE (2022)
12. Feldman, P.: A practical scheme for non-interactive verifiable secret sharing. In: *28th Annual Symposium on Foundations of Computer Science (sfcs 1987)*. pp. 427–438. IEEE (1987)
13. Fouque, P.A., Stern, J.: One round threshold discrete-log key generation without private channels. In: *Public Key Cryptography: 4th International Workshop on Practice and Theory in Public Key Cryptosystems, PKC 2001 Cheju Island, Korea, February 13–15, 2001 Proceedings 4*. pp. 300–316. Springer (2001)
14. Fournaris, A.P.: A distributed approach of a threshold certificate-based encryption scheme with no trusted entities. *Information Security Journal: A Global Perspective* **22**(3), 126–139 (2013)
15. Franklin, M., Yung, M.: Communication complexity of secure computation. In: *Proceedings of the twenty-fourth annual ACM symposium on Theory of computing*. pp. 699–710 (1992)
16. Galindo, D., Liu, J., Ordean, M., Wong, J.M.: Fully distributed verifiable random functions and their application to decentralised random beacons. In: *2021 IEEE European Symposium on Security and Privacy (EuroS&P)*. pp. 88–102. IEEE (2021)
17. Gennaro, R., Jarecki, S., Krawczyk, H., Rabin, T.: Secure distributed key generation for discrete-log based cryptosystems. In: *International Conference on the Theory and Applications of Cryptographic Techniques*. pp. 295–310. Springer (1999)

18. Gurkan, K., Jovanovic, P., Maller, M., Meiklejohn, S., Stern, G., Tomescu, A.: Aggregatable distributed key generation. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 147–176. Springer (2021)
19. Jhanwar, M.P.: A practical (non-interactive) publicly verifiable secret sharing scheme. In: Information Security Practice and Experience: 7th International Conference, ISPEC 2011, Guangzhou, China, May 30–June 1, 2011. Proceedings 7. pp. 273–287. Springer (2011)
20. Kalai, R., Neji, W., Ben Rajeb, N.: A distributed multi-key generation protocol with a new complaint management strategy. In: European, Mediterranean, and Middle Eastern Conference on Information Systems. pp. 150–164. Springer (2022)
21. Kalai, R., Neji, W., Ben Rajeb, N.: Reviewing the role of secret sharing schemes in electronic payment protocols. In: European, Mediterranean, and Middle Eastern Conference on Information Systems. pp. 52–60. Springer (2023)
22. Ma, T., Xu, H., Li, P.: A blockchain traceable scheme with oversight function. In: International Conference on Information and Communications Security. pp. 164–182. Springer (2020)
23. Ma, T., Xu, H., Li, P.: A traceable scheme for consortium blockchain. In: 2021 IEEE 9th International Conference on Smart City and Informatization (iSCI). pp. 39–46. IEEE (2021)
24. Neji, W., Blibech, K., Ben Rajeb, N.: Distributed key generation protocol with a new complaint management strategy. *Security and communication networks* **9**(17), 4585–4595 (2016)
25. Paillier, P.: Public-key cryptosystems based on composite degree residuosity classes. In: International conference on the theory and applications of cryptographic techniques. pp. 223–238. Springer (1999)
26. Pakniat, N., Noroozi, M., Eslami, Z.: Distributed key generation protocol with hierarchical threshold access structure. *IET Information Security* **9**(4), 248–255 (2015)
27. Pedersen, T.P.: Non-interactive and information-theoretic secure verifiable secret sharing. In: Annual international cryptology conference. pp. 129–140. Springer (1991)
28. Pedersen, T.P.: A threshold cryptosystem without a trusted party. In: Workshop on the Theory and Application of Cryptographic Techniques. pp. 522–526. Springer (1991)
29. Schindler, P., Judmayer, A., Stifter, N., Weippl, E.: Hydrand: Efficient continuous distributed randomness. In: 2020 IEEE Symposium on Security and Privacy (SP). pp. 73–89. IEEE (2020)
30. Schoenmakers, B.: A simple publicly verifiable secret sharing scheme and its application to electronic voting. In: Annual International Cryptology Conference. pp. 148–164. Springer (1999)
31. Shamir, A.: How to share a secret. *Communications of the ACM* **22**(11), 612–613 (1979)
32. Stadler, M.: Publicly verifiable secret sharing. In: International Conference on the Theory and Applications of Cryptographic Techniques. pp. 190–199. Springer (1996)
33. Syta, E., Jovanovic, P., Kogias, E.K., Gailly, N., Gasser, L., Khoffi, I., Fischer, M.J., Ford, B.: Scalable bias-resistant distributed randomness. In: 2017 IEEE Symposium on Security and Privacy (SP). pp. 444–460. Ieee (2017)
34. Wang, F., Chang, C.C., Harn, L.: Simulatable and secure certificate-based threshold signature without pairings. *Security and Communication Networks* **7**(11), 2094–2103 (2014)
35. Wong, T.M., Wang, C., Wing, J.M.: Verifiable secret redistribution for archive systems. In: First International IEEE Security in Storage Workshop, 2002. Proceedings. pp. 94–105. IEEE (2002)
36. Yin, M., Malkhi, D., Reiter, M.K., Gueta, G.G., Abraham, I.: Hotstuff: Bft consensus with linearity and responsiveness. In: Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing. pp. 347–356 (2019)
37. Yuan, H., Zhang, F., Huang, X., Mu, Y., Susilo, W., Zhang, L.: Certificateless threshold signature scheme from bilinear maps. *Information Sciences* **180**(23), 4714–4728 (2010)